CLUB DE VUELO UPM

Definición y esquemas para el Simulador

Especificaciones Técnicas

Pablo Martín Yuste

28 de septiembre de 2026

1. Aspectos generales del simulador

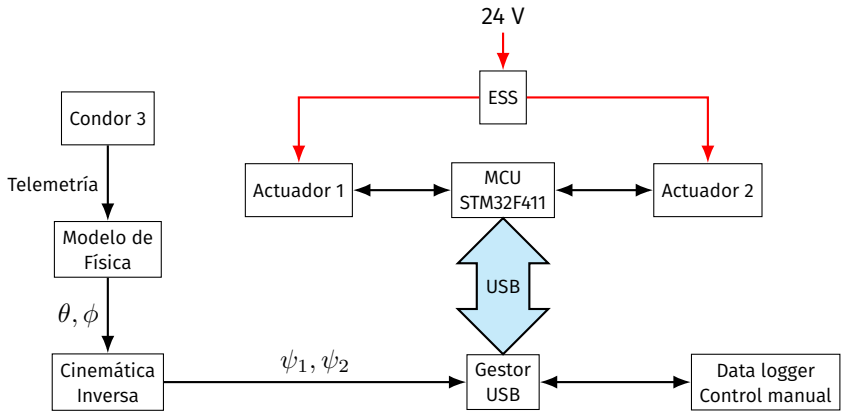
El simulador consiste en el fuselaje de la cabina de un planeador **ToDo 1:** Foto de un modelo del planeador del simu, montada sobre una junta universal que permite los giros de cabeceo y alabeo, y dotado de dos motores eléctricos que mueven las articulaciones a la parte posterior del fuselaje.

El software empleado es Condor 3, que permite una extracción de los datos del estado de vuelo del planeador mediante el protocolo UDP.

Este proyecto implementa un software para la extracción, interpretación y procesado de la telemetría de Condor 3, y lo combina con un hardware para controlar dinámicamente la actitud del planeador, con el objetivo de ofrecer una experiencia de vuelo realista.

2. Sistemas del simulador

El funcionamiento general del simulador se explica a continuación:



Como se puede observar, los datos brutos extraídos del Condor 3 pasan por dos tipos de procesado. En primer lugar la telemetría se convierte en unos ángulos de cabeceo y alabeo para el simulador real, utlizando un modelo de

física que busca reproducir de la forma más fiel posible las fuerzas que se sentirían en cabina en las condiciones del vuelo simulado.

A continuación, un cálculo de cinemática inversa resuelve a qué posiciones se deben mover cada uno de los motores para alcanzar esa posición. Una vez determinadas estas posiciones, estos datos son pasados al gestor USB mediante un ciclo de control de alta frecuencia, para mover los motores.

El microcontrolador es responsable de implementar un algoritmo de control **ToDo 2:** referencia cuando se explique este algoritmo para mover los motores. Cada uno de los motores está instrumentado con un sensor de ángulo magnético para conocer exactamente su posición, permitiendo un control en bucle cerrado.

Como medida de protección activa, está presente un botón de desconexión de emergencia (ESS, *Emergency Stop Switch*), que desactivará los puentes H e interrumpirá físicamente el suministro de corriente a los motores, permitiendo parar instantáneamente el movimiento del simulador. Además, el microcontrolador está conectado a este botón, indicando al software de escritorio esta detención.

2.1. Aplicación de escritorio

Para implementar de forma centralizada todo el código, se ha desarrollado una aplicación de escritorio responsable de preprocesar los datos de telemetría de Condor 3, y convertirlos en comandos de posición que enviar al sistema de control de los actuadores. Esta aplicación se ha desarrollado en Python, por sus facilidades para el desarrollo de la interfaz gráfica, con módulos individuales programados en C++. A continuación se explica detalladamente cada uno de sus componentes.

2.1.1. Interfaz Gráfica

Para facilitar el uso del software, se ha desarrollado una interfaz gráfica de uso general, que proporciona controles útiles, como activar o desactivar el sistema de movimiento, indicar si la telemetría se recibe correctamente de Condor 3, o controlar manualmente el simulador.

2.1.2. Algoritmo de Generación de Movimiento

ToDo 3: Escribir detalladamente el funcionamiento del modelo de vuelo

Se ha estudiado cómo implementar de la forma más satisfactoria el algoritmo de generación de movimiento para la plataforma del simulador. En primer lugar, se parte de alguno de los algoritmos de generación de movimiento genéricos. Después, se ha estudiado la forma más similar de adaptar la pose de una plataforma don 6 GDL a la del simulador, con solamente 2 GDL.

Actualmente, el principal problema consiste en resolver el “Método de Tustin”, que permite traducir las ecuaciones del MCA, originalmente en el dominio de la frecuencia, a ecuaciones algebraicas en un esquema de diferencias finitas.

2.1.3. Cinemática inversa

ToDo 4: Programar en C++ para reducir el tiempo de ejecución y aumentar la fiabilidad del código.

La cinemática inversa es el penúltimo proceso del cálculo de la posición de los actuadores. Consiste en, a partir de la actitud deseada para el planeador, resolver la posición y la velocidad de los actuadores.

Una explicación más detallada de este cálculo se puede encontrar en el Apéndice A.

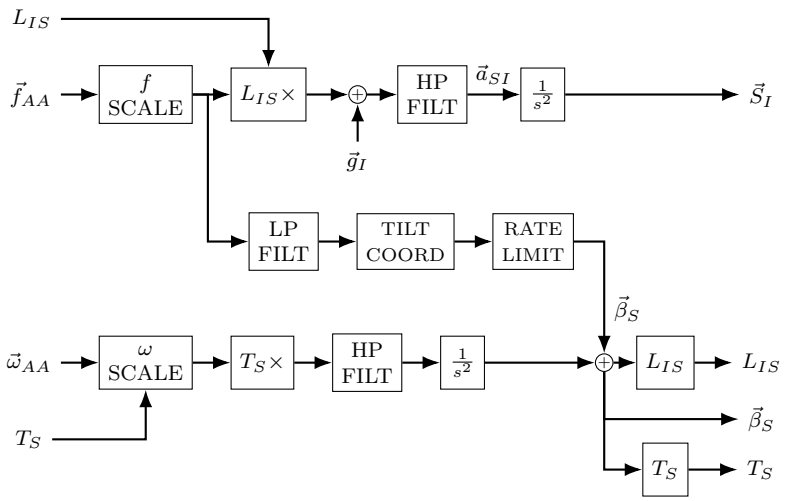


Figura 1: Representación del *Classical Washout*, de Nahon y Reid.

2.1.4. Gestor USB

Esta parte de la aplicación de escritorio desarrollada se encarga de procesar los datos que deben transmitirse por la comunicación USB y recibir la información de estado que el microcontrolador devuelve. Se ha programado de forma independiente para poder gestionar eficientemente altos volúmenes de información.

ToDo 5: Decidir si programar en Python o en C++.

2.1.5. Comunicación USB

ToDo 6: Definir comunicación, qué datos van desde la app de escritorio, qué datos manda el microcontrolador de vuelta, cuál es la latencia objetivo de la comunicación, etc.

2.1.6. Gestor de diagnósticos

Para asegurar el correcto funcionamiento del simulador, la aplicación de escritorio recibe información de diagnóstico del simulador y de Condor 3 para garantizar un funcionamiento correcto por parte de todos los sistemas interconectados. La información que recibe es:

- Códigos de estado por parte del microcontrolador (y códigos de error en caso de algún fallo).
- El estado de la telemetría del Cóndor 3.
- Si el simulador está en funcionamiento, calibrándose o listo para comenzar el movimiento.

2.1.7. Control manual

La plataforma del simulador se puede mover de forma manual, tanto con control individual de cada uno de los actuadores, como con control de la posición de la plataforma en cabeceo y alabeo.

2.2. Sistema de control del movimiento

Externo a la aplicación de escritorio, y utilizando la conexión USB como comunicación, el sistema de control del movimiento consiste en toda la electrónica encargada de garantizar el movimiento de los actuadores de forma

precisa y rápida. Los componentes se describen a continuación.

Se ha considerado como actuadores a todo el conjunto de puente H, motor y encoder magnético que está duplicado en el simulador. Cuando está conectado todo, el puente H recibe una señal PWM del microcontrolador que determina el par (y la velocidad) con la que el motor busca alcanzar la posición comandada. Para una representación general del funcionamiento de este subsistema, consultar la Figura 2

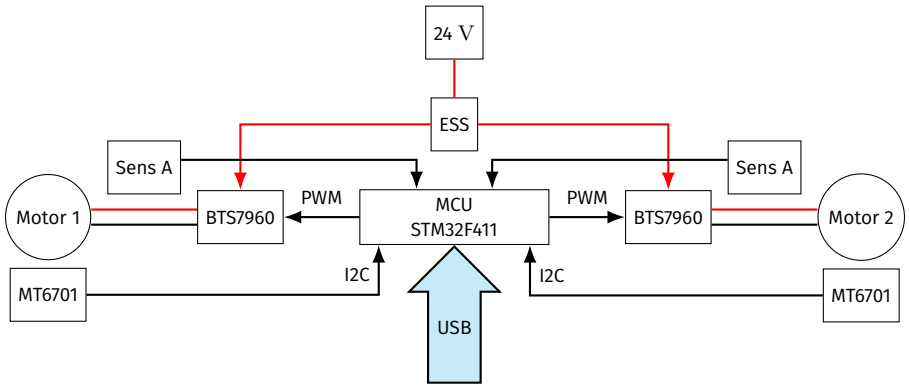


Figura 2: Funcionamiento general del sistema de control de movimiento

2.2.1. Microcontrolador

ToDo 7: Explicar qué código se ejecuta en el microcontrolador, y qué algoritmos se emplean para el control de los motores (actuadores), si se implementa (o no) la regulación de la intensidad máxima...

Se ha elegido el STM32F411 por su alta frecuencia del microprocesador, además de tener disponibles dos buses I2C independientes, permitiendo una comunicación de milisegundos con los encoders magnéticos en paralelo.



Figura 3: Foto del microontrolador STM32F411

2.2.2. Puentes H BTS7960-43A

Para poder conseguir un buen control de los motores, se utilizarán los puentes H BTS7960-43A para suministrar una tensión variable entre 0 y 24 V. Estos componentes reciben una señal PWM (Pulse Width Modulation) del microcontrolador, y la utilizarán para polarizar de forma directa o inversa el motor, dando pulsos de tensión con el mismo *duty cycle* que la señal PWM.
ToDo 8: Foto de los puentes h.

2.2.3. Motores

Los motores son actuadores del parabrisas de un autobús, extraídos de un desguace. Se trata de motores de contínua de 24 V. Son capaces de proporcionar 16 Nm de par, con una reducción mediante tornillo sinfín.

ToDo 9: Añadir foto de los motores

2.2.4. Encoders Magnéticos

Para determinar con precisión la posición de cada uno de los motores, se están utilizando encoders magnéticos MT6701. Estos se comunican mediante el protocolo I2C al microcontrolador, con una precisión de 14 bits en la comunicación.
ToDo 10: Foto de MT6701 montado y sin montar

Datasheet del sensor.

2.2.5. Fuentes de Alimentación

Las fuentes utilizadas en este sistema son las DPS-600 PB. Estas son unas fuentes con un diseño propietario, por lo que no hay disponibilidad de manual de uso. Sin embargo, su uso es popular entre aficcionados al radiocontrol, y hay mucha información disponible en foros.

ToDo 11: Foto de las fuentes de alimentación.
ToDo 12: Foto del pinout de las fuentes de alimientación.

3. Consideraciones

3.1. Control de Intensidad

Experiencias pasadas con este sistema, utilizando algoritmos de control menos sofisticados, han demostrado que es frecuente que se exceda la intensidad máxima de trabajo de los actuadores, durante cambios bruscos de sentido (inversiones de carga). Esto puede provocar el colapso de los puentes H o daños en los devanados de los motores, por lo que es recomendable implementar instrumentación de medida de corriente, para añadir regulación en un futuro.

3.2. Resensorización del simulador

En el estado actual del simulador, las palancas físicas responsables del movimiento de los ejes de control está siendo gestionado por dos microcontroladores independientes: un LEO-BODNAR (antiguo) y un Arduino Pro Micro (más moderno). El LEO-BODNAR se encarga de los interruptores del tren de aterrizaje, mientras que el Arduino se encarga de los ejes de freno, guiñada y compensador.

Para evitar cableado y sistemas extra que no son necesarios, una posible mejora sería eliminar el controlador LEO-BODNAR y utilizar el Arduino Pro Micro para controlar todos los botones y ejes de la cabina. Además, los ejes cuya medida se hace ahora mediante potenciómetros, se deberían cambiar a sensores magnéticos de efecto de campo o encoders magnéticos, para aumentar la precisión y la vida de estos sensores.

La mejor propuesta para hacer esto consistiría en utilizar el multiplexor TCA9548A, conectado al Arduino para tener hasta 8 canales I2C con los que leer datos de encoders magnéticos, y considerar utilizar alguna de las matrices para botones que hay disponibles en el club.

Lista de cosas pendientes

1. Foto de un modelo del planeador del simu ..... p. 1
2. referencia cuando se explique este algoritmo ..... p. 2
3. Escribir detalladamente el funcionamiento del modelo de vuelo ..... p. 2
4. Programar en C++ para reducir el tiempo de ejecución y aumentar la fiabilidad del código. ..... p. 2
5. Decidir si programar en Python o en C++ ..... p. 3
6. Definir comunicación ..... p. 3
7. Explicar qué código se ejecuta en el microcontrolador ..... p. 4
8. Foto de los puentes h ..... p. 4
9. Añadir foto de los motores ..... p. 5
10. Foto de MT6701 montado y sin montar ..... p. 5
11. Foto de las fuentes de alimentación ..... p. 5
12. Foto del pinout de las fuentes de alimientación ..... p. 5

A. Resolución de la cinemática inversa

La resolución de la cinemática inversa se hace en varios pasos. A continuación se explica el proceso para un actuador, pero evidentemente el cálculo se hace por duplicado al haber dos actuadores.

1. Con los ángulos θ y ϕ obtenidos del modelo de vuelo, se calcula la matriz de rotación que permite convertir vectores de ejes cuerpo a ejes tierra. Para obtener esta matriz, se ha empleado el criterio mostrado en la Figura 4

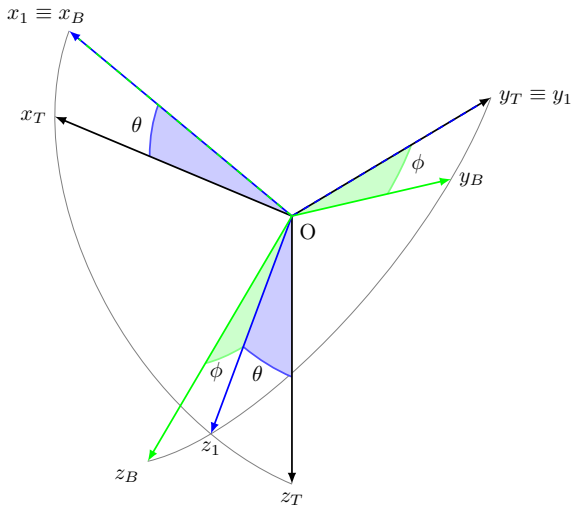


Figura 4: Diagrama de la rotación mediante ángulos de Euler

2. Se calcula la posición final de los puntos J, los extremos de la barra horizontal unida al fuselaje a la que se unen las barras de las articulaciones. Con esto, se calcula la distancia del centro de cada motor (puntos M) a las juntas. Esta distancia será l.

3. Con esto, los tres lados del triángulo formado por los lados l, s y t son conocidos, de forma que se puede emplear el teorema del coseno para calcular el ángulo ($\hat{S}$) que hay entre el lado l y el lado t.

4. A partir de las componentes l_x y l_z del vector $\vec{l}$, se puede calcular la longitud que tendrá el lado l', que será la proyección del lado l del triángulo en el plano XZ. De este triángulo, el lado t también es conocido, porque este mismo lado está contenido siempre en el plano XZ.

5. Aprovechando que la prependicularidad entre b, el segmento que es perpendicular a l y pasa por el otro extremo de t, se conserva entre proyecciones[1], se puede resolver el triángulo $tp'b'$, ya que se conocen dos lados y un ángulo (ver Figura 5)

6. Con el triángulo $b'p't$ resuelto, se puede calcular finalmente el ángulo que formará t con la horizontal, resolviendo finalmente el ángulo ψ del actuador.

[1] Esto no es una propiedad de la proyección, sino una particularidad por estar proyectando un triángulo que tiene un lado contenido en el plano XZ a ese mismo plano.

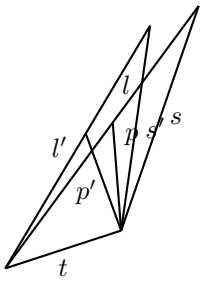


Figura 5: Representación de los triángulos lst, btp y $b'p't$.